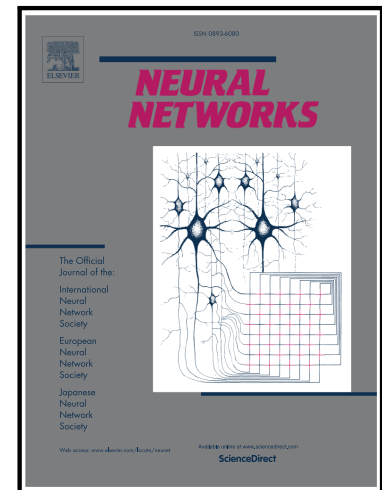


KD-GAG: Knowledge-Distilled Graph-Augmented Generation via Preference-Aware Subgraph Pruning

Long Zhao, Yin Xu, Yanyan Wang, Fei Li

PII: S0893-6080(26)01044-0
DOI: <https://doi.org/10.1016/j.neunet.2026.109587>
Reference: NN 109587



To appear in: *Neural Networks*

Received date: 26 January 2026
Revised date: 24 August 2026
Accepted date: 2 September 2026

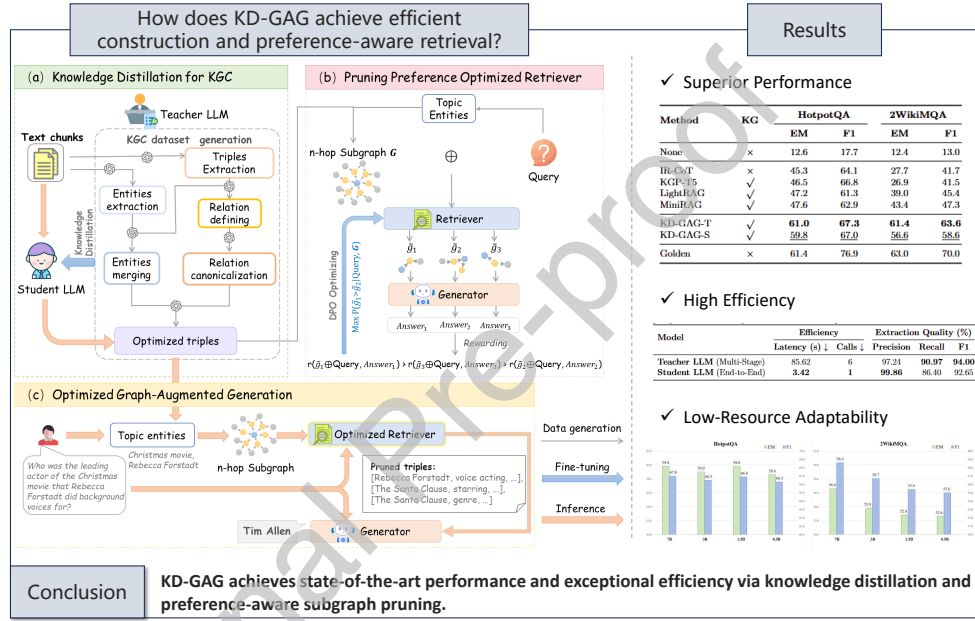
Please cite this article as: Long Zhao, Yin Xu, Yanyan Wang, Fei Li, KD-GAG: Knowledge-Distilled Graph-Augmented Generation via Preference-Aware Subgraph Pruning, *Neural Networks* (2026), doi: <https://doi.org/10.1016/j.neunet.2026.109587>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Graphical Abstract

KD-GAG: Knowledge-Distilled Graph-Augmented Generation via Preference-Aware Subgraph Pruning

Long Zhao, Yin Xu, Yanyan Wang, Fei Li



Highlights

KD-GAG: Knowledge-Distilled Graph-Augmented Generation via Preference-Aware Subgraph Pruning

Long Zhao, Yin Xu, Yanyan Wang, Fei Li

- Proposes KD-GAG for efficient KG construction and precise retrieval.
- Optimizes retrieval by aligning with generator’s data preferences.
- Outperforms baselines in efficiency while maintaining accuracy.

KD-GAG: Knowledge-Distilled Graph-Augmented Generation via Preference-Aware Subgraph Pruning

Long Zhao^{a,b}, Yin Xu^b, Yanyan Wang^{b,*}, Fei Li^{a,b}

^a*University of Science and Technology of China, School of Computer Science and Technology, No.100, Fuxing Road, High tech Zone, Hefei, 230026, Anhui, China*

^b*GuoChuang Cloud Technology Ltd., Innovation & Research Institute, No.355 Wenqu Road, High tech Zone, HeFei, 230031, Anhui, China*

Abstract

Retrieval-Augmented Generation (RAG) methods that integrate external knowledge sources have demonstrated significant effectiveness in addressing the knowledge scarcity and hallucination issues of Large Language Models (LLMs). Compared to traditional approaches relying on documents as knowledge sources, Knowledge Graphs (KGs) enable GraphRAG to emerge as a superior solution through their capability to capture complex interdependencies. However, existing GraphRAG methods still exhibit limitations: insufficient attention to optimizing KG construction, introduction of excessive irrelevant information during recall optimization in retriever, and inadequate consideration of the generator’s data preferences during retrieval refinement. In this paper, we propose a novel framework KD-GAG that efficiently constructs KGs by distilling teacher LLMs’ semantics and reasoning into a smaller student LLM. Furthermore, we propose a preference-based subgraph pruning method to optimize the retrieval process. This method automatically learns subgraph pruning strategies through answer quality feedback and aligns the retriever’s selection mechanism with the generator’s knowledge preferences. Extensive experiments on four QA benchmarks, including HotpotQA, 2WikiMQA, NQ, and MedHop, demonstrate that KD-GAG achieves strong and consistent performance across multi-hop QA, open-domain QA, and biomedical reasoning settings.

Keywords: Retrieval-Augmented Generation, Knowledge Graph, Question Answering

*Corresponding author.

1. Introduction

Retrieval-Augmented Generation (RAG) has attracted significant attention in recent years due to enhancing generation quality of Large Language Models (LLMs) by integrating external knowledge sources [1, 2]. As shown in Figure 1, by retrieving relevant information from large-scale document collections, RAG effectively alleviates the knowledge limitations and hallucination issues of LLMs [3]. However, current RAG systems [4, 5, 6] still face several critical challenges. First, these models may struggle to fully understand contextual semantics, leading to imprecise retrieval results. Second, the retrieved contexts often contain substantial noise, which interferes with the model’s reasoning and generation processes [7]. Lastly, when dealing with massive datasets, traditional retrieval methods suffer from low efficiency, making it difficult to meet the requirements of real-time and large-scale applications. These issues constrain the practical performance of RAG models in real-world scenarios.

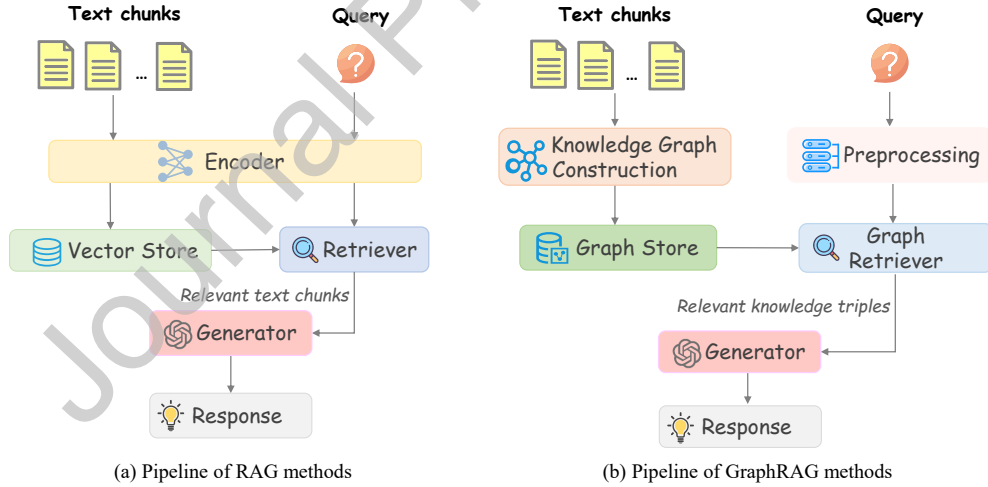


Figure 1: Comparison of Pipelines for (a) RAG Methods and (b) GraphRAG Methods.

Inspired by the potential of graph-based structures, current research focuses on Graph-based RAG methods [8, 9, 10, 11], which enhance retrieval

efficiency and knowledge representation by storing knowledge in graph structures. Nevertheless, existing GraphRAG approaches still exhibit notable shortcomings. Firstly, regarding knowledge graph construction, many methods [8, 9] rely on existing knowledge graphs, lacking customization for specific domain document collections. Even when some methods attempt to construct knowledge graphs from documents [10, 11], their focus is often on optimizing the retrieval module, resulting in overly simplified knowledge graph construction processes that fail to accurately extract complete knowledge. Moreover, existing unsupervised knowledge graph construction methods typically require multiple calls to LLMs, leading to inefficiency and hindering their applicability to large-scale datasets. Furthermore, while existing methods optimize the retriever, they overlook the alignment of data preferences between the retriever and the generator, limiting the generator’s ability to fully leverage the retrieved knowledge and thus failing to maximize its value.

To tackle these challenges, we propose a novel framework, KD-GAG, aimed at comprehensively enhancing the knowledge extraction and retrieval capabilities of GraphRAG models. Specifically, to balance the efficiency and quality of knowledge graph construction, we introduce a knowledge distillation-based method. A multi-step knowledge extraction and fusion process is performed using a large-scale Teacher LLM to generate high-quality knowledge graphs as training data, which are then used to fine-tune a smaller Student LLM, enabling efficient end-to-end knowledge graph extraction. Additionally, to align the data preferences of the generator and retriever, we calculate rewards based on the generator’s final responses to perform preference optimization training for the retriever, ensuring better synergy between the two components.

A key design issue in KD-GAG is how to preserve useful graph evidence while removing irrelevant triples. Unlike conventional passage retrieval, subgraph pruning operates on structured triples connected by entities and relations, where the utility of each triple depends not only on its local relevance to the question but also on its role in the reasoning structure captured by the selected subgraph. In addition, most QA datasets provide only answer-level supervision rather than explicit labels indicating which triples should be retained. Therefore, directly treating triples as independent preference units is inadequate for this setting. KD-GAG instead constructs preferences over candidate subgraphs and evaluates them through their downstream reasoning utility. Preference pairs with small utility differences are filtered out before optimization, reducing ambiguous preference supervision. In this way,

the pruning retriever learns structured evidence selection from downstream reasoning effectiveness without requiring triple-level annotations.

We highlight our contributions as follows:

1. We develop a knowledge distillation scheme for efficient open-schema knowledge graph construction, which compresses the Teacher LLM’s multi-stage extraction, relation definition, and canonicalization pipeline into an end-to-end Student LLM using quality-filtered graph supervision. This design reduces repeated LLM invocations while preserving structured knowledge for downstream GraphRAG.

2. We introduce a preference-aware graph retriever optimization method tailored to structured subgraph pruning in GraphRAG. Instead of treating triples as independent relevance units, the method constructs candidate subgraphs, evaluates their downstream reasoning utility through answer-level feedback, and optimizes the retriever with margin-filtered preference pairs. This design adapts preference optimization to graph-structured evidence selection without requiring triple-level supervision.

3. Extensive experiments on four QA benchmarks, including multi-hop QA, open-domain QA, and biomedical reasoning, demonstrate the effectiveness and adaptability of KD-GAG. Competitive results are achieved even when employing the Student LLM for knowledge graph construction.

2. Related Work

KG+RAG. RAG enhances LLM generation by retrieving relevant knowledge from external knowledge bases, alleviating issues of knowledge scarcity and hallucination. For instance, methods such as IR-CoT [6] and KGP-T5 [12] optimize the retrieval module and incorporate top-k retrieved text chunks into prompt to generate more accurate responses. However, RAG systems that use documents as knowledge bases may introduce significant noise from excessive context, which can negatively impact the model’s reasoning performance. Recently, several studies [10, 11] have improved corpus comprehension and retrieval efficiency by storing knowledge in graph structures [13]. More recent methods, such as HippoRAG 2 [14] and LinearRAG [15], further improve structure-aware retrieval by enhancing graph-based memory mechanisms or simplifying graph retrieval pipelines. Nevertheless, challenges persist regarding inefficient knowledge graph construction and insufficient relevance of retrieved information. Alternatively, our proposed KD-GAG accelerates the construction of knowledge graphs through knowledge distillation

and optimizes the retriever’s preference for relevant knowledge by calculating rewards based on the final response, which enables the filtered knowledge to better facilitate ultimate generation.

Knowledge Distillation. Knowledge distillation is a critical technique for model compression and acceleration [16, 17], focusing on transferring knowledge from a large teacher model to a smaller student model. Recent methods can be divided into white-box and black-box approaches [18], where white-box methods utilize internal parameters, intermediate features, or gradients of the teacher model to guide knowledge transfer, while black-box methods rely solely on the teacher’s outputs to generate data for fine-tuning the student model. Although knowledge distillation has achieved success in many tasks, such as classification [19] and instruction following [20], it still faces challenges in balancing efficiency and quality for knowledge graph construction. Therefore, we propose a distillation framework optimized for knowledge graph construction that uses multi-step reasoning and data generation to produce high-quality training samples preserving the teacher model’s deep semantic information while reducing noise, followed by probabilistic distribution alignment for efficient student model fine-tuning, ensuring high performance and computational efficiency.

Reinforcement Learning. In reinforcement learning for language model alignment, preference-based optimization has advanced significantly [21, 22]. While early approaches like Proximal Policy Optimization (PPO) [23] required explicit reward models and online sampling, they faced computational complexity and instability. Direct Preference Optimization (DPO) [24] circumvented these limitations by reformulating preference learning as a supervised classification task on response pairs, proving that KL-divergence-constrained reward maximization has closed-form solutions. DPO has shown state-of-the-art results in dialogue generation [25] and summarization [26]. Inspired by DPO, we optimize the retriever’s parameters by maximizing the selection probability of relevant triples, thereby enhancing triple selection, avoiding overfitting issues from traditional fine-tuning [27], and improving retriever efficiently and stably with strong generalization.

3. Preliminaries

Let $\mathcal{D} = \{\mathcal{D}_1, \dots, \mathcal{D}_{|\mathcal{D}|}\}$ denote a set of documents, where each document $\mathcal{D}_i$ consists of passages: $\mathcal{D}_i = \{d_1, \dots, d_{|\mathcal{D}_i|}\}$. Through knowledge triple ex-

traction applied to all passages in $\mathcal{D}$, we obtain a structured set of triples:

$$\mathbf{T} = \{t_1, \dots, t_{|\mathbf{T}|} : t_j = (\text{head}_j, \text{rel}_j, \text{tail}_j)\} \quad (1)$$

where head_j , rel_j and tail_j represent the head entity, relation and tail entity of the j -th triple, respectively.

Given an input query q , the first step is retrieving relevant triplets $g = \{t_1, \dots, t_{|g|} | t_k \in \mathbf{T}\}$ from the knowledge graph by the retriever: $g = \text{Retriever}(q, \mathbf{T})$. The retrieved triples g are then concatenated with q and fed into the generator, which performs knowledge-augmented reasoning to produce the final response: $\text{response} = \text{Generator}(q, g)$.

4. Methodology

4.1. Overview

As illustrated in Figure 2, KD-GAG comprises three key modules. First, to reduce the number of LLM invocations and establish an end-to-end knowledge graph construction workflow, we employ a Teacher LLM to generate high-quality data for fine-tuning a smaller Student LLM, efficiently transferring the knowledge extraction capability while significantly reducing computational resources. Then, to accurately retrieve triples relevant to the given query, we extract topic entities from the query and retrieve an n -hop subgraph centered around these entities. This subgraph is subsequently pruned using our designed retriever optimized via reinforcement learning, retaining only the most relevant information. Finally, during the question answering stage, both the query and the retrieved knowledge triples are fed into the generator, which reasons over the provided information to infer the answer.

4.2. Knowledge Distillation for Knowledge Graph Construction

Knowledge Graph Construction. We mainly follow [28] to extract knowledge triples from the documents, which is divided into three stages. In the first stage, structured information is autonomously identified and extracted from the input text without relying on any predefined schema. This is achieved through the use of few-shot prompting, where LLM is instructed to extract knowledge triples directly from the input text.

In the second stage, to enhance the LLM’s understanding of relationships between entities, few-shot prompting is used to generate definitions for the relations extracted in the previous stage. If a target relation exists, an attempt is made to match the extracted relations with existing types within

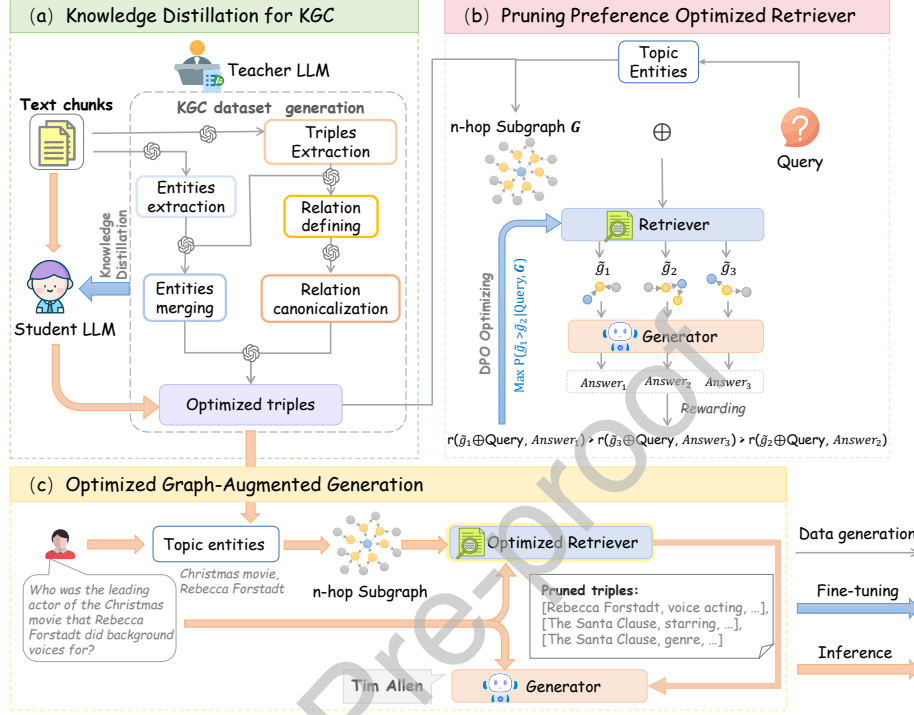


Figure 2: Overview of our KD-GAG framework. The blue arrows indicate the reasoning process, the gray arrows represent the data collection procedure, and the orange arrows denote the fine-tuning training phase.

the schema. In cases where no predefined schema is available, new relation types are dynamically created to ensure a consistent schema structure.

Finally, in the third stage, the entities and relations within the extracted triples are further standardized. This process leverages the previously extracted entities and relations as references, allowing the model to perform re-extraction iteratively. This approach reduces redundancy and improves the overall consistency of the knowledge graph. Through these three stages, the method ensures robust and adaptable knowledge graph construction, capable of accommodating both predefined and dynamically generated schema.

Knowledge Distillation. The aforementioned knowledge graph construction method eliminates the need for predefined schema while maintaining flexible extraction and comprehensive graph completeness. However, it suffers from inefficiency due to multiple LLM invocations and potential error

propagation through intermediate processing stages. To address these limitations, we propose a knowledge distillation framework that simplifies the multi-step large LLM pipeline into an end-to-end extraction process using a smaller-scale Student LLM. We generate supervised training data by applying the original method with the Teacher LLM, though the dataset quality is inherently affected by LLM hallucination issues. To ensure data reliability, we implement dual-aspect filtering criteria focusing on completeness and accuracy:

- Completeness Criteria 1: All intermediate outputs strictly follow the specified formatting requirements.
- Completeness Criteria 2: Each extracted triple consists of a head entity, a relation, and a tail entity, forming complete components.
- Accuracy Criteria 1: No semantically redundant elements exist within final extraction results.
- Accuracy Criteria 2: All triple components either explicitly appear in source text or have contextually equivalent paraphrases.

The refined high-quality dataset is subsequently utilized to fine-tune the Student LLM, achieving significant efficiency improvements in extraction tasks. While minor error margins persist, our distilled approach demonstrates superior balance between processing efficiency and result quality compared to the original methodology. This optimized solution provides a more practical and efficient paradigm for knowledge graph construction, particularly suitable for real-world deployment scenarios requiring both accuracy and computational efficiency. Importantly, the distilled Student LLM is used only for upstream knowledge graph construction and is not tied to any specific downstream generator. In addition, its open-schema extraction paradigm avoids reliance on manually predefined schemas, making the distillation framework adaptable to new domains when in-domain distillation data is available. The additional evaluation on MedHop further demonstrates that the framework can be adapted to biomedical multi-hop reasoning, where domain-specific distillation data helps capture specialized terminology and relation patterns.

4.3. Pruning Preference Optimized Retriever

Subgraph Retrieving. To enhance the retrieval of more relevant information from the knowledge graph, our retrieval process initiates with topic

entity recognition from the input query. This step is accomplished by a LLM with few-shot prompt. Subsequently, these identified topic entities are mapped onto the knowledge graph through semantic similarity. Following the mapping process, the Breadth-First Search (BFS) [29] algorithm is employed to retrieve an n -hop subgraph $\mathcal{G}$ centered around these entities. The answer-supporting evidence is likely to reside within $\mathcal{G}$. However, using the entire subgraph as direct input may result in excessive verbosity, which could impair the generator’s ability to process and comprehend the information effectively. Additionally, the presence of redundant information may introduce noise, negatively impacting the generator’s reasoning performance. Therefore, it is crucial to apply a pruning process to refine and optimize the information, ensuring that only relevant and concise data is presented for efficient reasoning and inference.

Subgraph Pruning. Existing methods [8, 9] mainly rely on LLMs to either directly predict reasoning paths or use models fine-tuned for this task, followed by retrieval in the knowledge graph to obtain answers. Although these approaches leverage the semantic understanding capabilities of LLMs to simplify the retrieval process, they often struggle to ensure that the generated paths actually exist within the graph due to the hallucination issue of LLMs. Additionally, the lack of training data for intermediate steps in most datasets limits the applicability of these methods. To address these limitations, we propose a pruning approach based on reinforcement learning, which does not require labeled intermediate data and can provide highly relevant knowledge for subsequent generation.

Adapting preference optimization to subgraph pruning is non-trivial because the optimization target is a structured subset of interconnected triples rather than independent passages or isolated text outputs. Moreover, explicit supervision indicating which triples should be retained is generally unavailable. We therefore construct preferences at the subgraph level by sampling candidate subgraphs from the retrieved graph, evaluating the downstream reasoning utility of each candidate through answer-level feedback from the generator, and retaining only sufficiently discriminative preference pairs for optimization. This allows structured evidence selection to be learned without triple-level annotations.

While LLMs can serve as a base retriever to perform pruning directly, they may misjudge weakly related triples. Specifically, we first use an initial LLM-based retriever Retriever_θ to sample a set of diverse pruned subgraph candidates. To ensure diversity in the sampled reasoning paths, we employ

temperature sampling with temperatures ranging from 0.5 to 1.2:

$$\mathcal{G}_{cand} = \{\text{Retriever}_\theta(\text{Prompt}_{prune}, \mathcal{G}, q; \tau) \mid \tau \in \{0.5, 0.6, \dots, 1.2\}\} \quad (2)$$

where $\mathcal{G}_{cand}$ denotes the set of candidate pruned subgraphs.

To evaluate the quality of each candidate $\tilde{g} \in \mathcal{G}_{cand}$, we rely on the feedback from the downstream generator. Each candidate is concatenated with the query q and fed into the fixed Generator to produce a predicted answer. We then calculate the F1 score between the predicted answer and the ground truth as the reward signal $R(\tilde{g})$:

$$R(\tilde{g}) = \text{F1}(\text{Generator}(q, \tilde{g}), \text{Answer}_{gt}) \quad (3)$$

Based on these rewards, we construct preference pairs $(\tilde{g}^+, \tilde{g}^-)$ for optimization. The candidate with the highest F1 score is selected as the positive sample $\tilde{g}^+$, and the one with the lowest score is selected as the negative sample $\tilde{g}^-$. To ensure the stability of optimization, we explicitly filter out pairs with low discriminability. A pair is included in the training set only if the performance gap exceeds a strict threshold δ (i.e., $R(\tilde{g}^+) - R(\tilde{g}^-) > \delta$). In our experiments, we set $\delta = 0.5$ to guarantee that the positive subgraph provides significantly better reasoning support than the negative one.

With the constructed preference pairs, we optimize the retriever parameters ϕ to maximize the likelihood margin between the positive and negative subgraphs. The training objective is minimized via the following DPO loss:

$$\mathcal{L}_{\text{DPO}}(\phi) = -\mathbb{E}_{(\mathcal{G}, q, \tilde{g}^+, \tilde{g}^-)} \left[\log \sigma \left(\beta \log \frac{P_\phi(\tilde{g}^+ \mid \mathcal{G}, q)}{P_\theta(\tilde{g}^+ \mid \mathcal{G}, q)} - \beta \log \frac{P_\phi(\tilde{g}^- \mid \mathcal{G}, q)}{P_\theta(\tilde{g}^- \mid \mathcal{G}, q)} \right) \right] \quad (4)$$

Through this process, the original LLM-based retriever is fine-tuned into a more effective model Retriever_ϕ , which is then used for the final retrieval.

4.4. Graph-Augmented Generation

Following the retrieval process, the query q and the relevant triples $\tilde{g}$ are fed into the generator to reason out the answer:

$$\text{response} = \text{Generator}(\text{Prompt}_{QA}, \tilde{g}, q) \quad (5)$$

where Prompt_{QA} is the prompt template designed for graph-augmented generation.

5. Experiment

We conduct experiments on benchmark datasets to demonstrate the effectiveness of our method by answering the following research questions:

- RQ1: Does our proposed method outperform existing approaches on benchmark datasets?
- RQ2: Can the proposed knowledge distillation method and retriever optimization method enhance the model’s performance?
- RQ3: Does the distilled Student LLM serve as an efficient and viable alternative to the Teacher LLM for large-scale knowledge graph construction in resource-constrained scenarios?
- RQ4: Does our preference-aware retriever improve retrieval quality beyond semantic denoising?
- RQ5: How does the selection of the student LLM for knowledge distillation impact the final results?
- RQ6: How strongly does the preference-aware retriever depend on the specific generator used during optimization, and can it generalize across different models?

5.1. Experimental Settings

Evaluation Datasets. We conduct experiments across diverse QA benchmarks to comprehensively evaluate our proposed method. For complex multi-hop reasoning, we utilize two widely-used datasets: **HotpotQA** [30] and **2WikiMQA** [31]. Both datasets require aggregating information across multiple documents to answer complex queries. Specifically, we evaluate under the distractor setting, where the model is provided with a mix of supporting facts and irrelevant documents. This setup serves as an ideal testbed to verify our model’s capability in distinguishing relevant knowledge from noise through precise subgraph pruning. Furthermore, to demonstrate the generalizability and robustness of our approach across different tasks and domains, we incorporate the **Natural Questions (NQ)** [32] and **MedHop** [33] datasets. NQ is a standard open-domain QA dataset comprising real-world user search queries, which evaluates the model’s performance in single-hop, information-seeking scenarios. MedHop is a specialized multi-hop

QA dataset in the biomedical domain, requiring the model to reason over complex interactions between drugs and proteins across multiple documents.

Baselines. **None** represents no passages but only the query is provided, while **Golden** represents that supporting facts are provided along with the query. **IR-CoT** [6] employs a chain-of-thought approach for retrieval, aimed at optimizing multi-hop question answering. **KGP-T5** [12] constructs a knowledge graph with passages or document structures serving as nodes, and utilizes a LLM-based graph traversal agent to selectively access nodes pertinent to the query. **LightRAG** [10] implements a two-tier retrieval architecture based on a knowledge graph, enhancing query understanding by extracting both low-level and high-level keywords from the query to retrieve corresponding relevant information within the knowledge graph. **MiniRAG** [11] on the other hand, accomplishes knowledge graph-based retrieval augmented RAG through a heterogeneous graph index and a lightweight heuristic retrieval mechanism. **HippoRAG 2** [14] incorporates a neurobiologically inspired memory architecture, leveraging personalized PageRank algorithms over a constructed knowledge graph to simulate human associative memory for multi-hop retrieval. **LinearRAG** [15] streamlines the graph retrieval pipeline by linearizing complex graph structures, thereby optimizing retrieval efficiency and achieving linear-complexity knowledge access while preserving the underlying relational semantics.

Evaluation Metrics. Following previous work, we employ Exact Match (EM) and F1 score to evaluate QA performance on HotpotQA, 2WikiMQA, and NQ, and use accuracy for MedHop. These metrics assess the correctness of the generated answers compared with the ground truth. In addition to end-to-end QA evaluation, we further conduct a retrieval-side analysis in RQ4. Since these datasets do not provide human-annotated triple-level labels, we construct pseudo-labeled essential triples and evaluate retrieval quality using Recall and Answer Hit Rate.

Implementation Details. We employ GPT-3.5-turbo as the Teacher LLM. The Student LLM, Retriever backbone, and Generator are all based on Qwen2.5-7B-Instruct. Crucially, to ensure a fair comparison and isolate the contribution of the retrieval module, we utilize the exact same Generator across all baseline methods and our KD-GAG. For subgraph retrieval, the hop size n is dynamically set to 3 for multi-hop datasets and 1 for single-hop datasets. For training configurations, knowledge distillation is performed for 2 epochs with a batch size of 4 and a learning rate of $2e-5$. The DPO-based Retriever optimization is trained for 2 epochs with a batch size of 8, a learning

rate of $2e-5$, and the KL penalty parameter β is set to 0.15. Both fine-tuning processes utilize LoRA [34] for efficient training on 4xA100-40G GPUs, with LoRA rank $r = 8$, $\alpha = 32$, and targeting the q_proj and v_proj modules. During the inference phase, the Generator decodes with a temperature of 0.1 and a maximum of 512 generated tokens. All implementations are based on the HuggingFace Transformers and TRL libraries.

5.2. Overall Performance (RQ1)

Table 1 shows the performance comparison of our method against various baselines on HotpotQA, 2WikiMQA, NQ, and MedHop. Overall, KD-GAG achieves the best performance among all compared methods excluding the Golden setting. Specifically, KD-GAG-T obtains the best results on HotpotQA and 2WikiMQA, while KD-GAG-S achieves the best results on NQ and MedHop. Compared with the strongest baseline on each dataset, KD-GAG improves EM by 3.2, 15.2, and 19.4 percentage points on HotpotQA, 2WikiMQA, and NQ, respectively, and improves accuracy by 2.6 percentage points on MedHop. These results demonstrate that KD-GAG achieves consistently strong performance across different QA settings, including multi-hop QA, open-domain QA, and biomedical reasoning.

The improvements of KD-GAG show different characteristics across datasets. On HotpotQA, strong baselines such as HippoRAG 2 and KGP-T5 already achieve competitive results, so the improvement of KD-GAG is relatively moderate but still consistent. On 2WikiMQA, KD-GAG brings a much larger improvement, suggesting that fine-grained entity-relation triples and preference-aware subgraph pruning are particularly effective when precise evidence selection is required. On NQ, KD-GAG-S substantially outperforms all baselines, indicating that the proposed graph-based evidence organization and pruning strategy can also benefit open-domain QA by reducing irrelevant context. On MedHop, although all methods obtain relatively low absolute accuracy due to the difficulty of biomedical reasoning, KD-GAG-S still achieves the best result, showing the adaptability of our framework to specialized domains.

The comparison between KD-GAG-T and KD-GAG-S further validates the effectiveness of knowledge distillation for knowledge graph construction. Although KD-GAG-S uses the distilled Student LLM instead of the Teacher LLM, it achieves performance close to KD-GAG-T on HotpotQA and 2WikiMQA, and even slightly outperforms KD-GAG-T on NQ and MedHop. This indicates that the Student LLM can preserve the essential knowledge

Table 1: Overall evaluation results on HotpotQA, 2WikiMQA, NQ, and MedHop. The **best** and second-best results are highlighted (excluding the Golden setting). KD-GAG-T refers to the method where the Teacher LLM is used for knowledge graph construction, while KD-GAG-S uses the distilled Student LLM.

Method	KG	HotpotQA		2WikiMQA		NQ		MedHop
		EM	F1	EM	F1	EM	F1	Acc
None	×	12.6	17.7	12.4	13.0	21.6	24.6	0.0
IR-CoT	×	45.3	64.1	27.7	41.7	22.4	25.1	9.1
KGP-T5	✓	46.5	66.8	26.9	41.5	9.8	11.6	11.1
LightRAG	✓	47.2	61.3	39.0	45.4	31.2	34.4	11.7
MiniRAG	✓	47.6	62.9	43.4	47.3	30.4	33.4	12.3
HippoRAG 2	✓	57.8	64.0	45.4	50.2	31.4	34.2	15.5
LinearRAG	✓	53.6	59.9	46.2	53.8	26.0	29.3	17.0
KD-GAG-T	✓	61.0	67.3	61.4	63.6	<u>48.8</u>	<u>52.3</u>	<u>18.4</u>
KD-GAG-S	✓	<u>59.8</u>	<u>67.0</u>	<u>56.6</u>	<u>58.6</u>	50.8	54.9	19.6
Golden	×	61.4	76.9	63.0	70.0	57.2	61.8	12.6

extraction capability required for downstream QA. Meanwhile, since the Student LLM performs end-to-end triple extraction with lower inference cost, KD-GAG-S provides a more efficient alternative for large-scale knowledge graph construction.

Among the baselines, recent methods such as HippoRAG 2 generally perform better than earlier approaches on several datasets, reflecting the importance of stronger retrieval and reasoning strategies. However, these methods still lag behind KD-GAG in most settings, especially on 2WikiMQA and NQ. This suggests that simply improving retrieval or introducing graph structures is not sufficient. The advantage of KD-GAG comes from the combination of high-quality knowledge graph construction and preference-aware subgraph pruning, which helps select answer-supporting evidence that is more suitable for the downstream generator. The None setting further confirms the necessity of external knowledge, while the Golden setting provides a reference result using gold contexts.

5.3. Ablation Study (RQ2)

To evaluate the effectiveness of key modules in KD-GAG, we conduct ablation experiments on knowledge distillation and the optimized retriever, as shown in Table 2. First, we remove the distillation process and directly use the Student LLM to construct knowledge graphs following the same procedure as the Teacher LLM. The results show consistent performance drops across all datasets. Compared with the full KD-GAG model, EM decreases by 15.2, 12.4, and 17.2 percentage points on HotpotQA, 2WikiMQA, and NQ, respectively, while accuracy on MedHop decreases from 19.6 to 17.1. This indicates that knowledge distillation improves the Student LLM’s ability to construct useful knowledge graphs for downstream retrieval and question answering.

Secondly, to validate our optimized retriever, we designed five different retrieval strategies for comparative analysis: 1) **N-hop subgraph** uses the retrieved n-hop subgraph centered around topic entities as input knowledge; 2) **Top-5 triples by BM25** selects the top-5 triples from the n-hop subgraph using BM25 [35]; 3) **5 triples by random walk** chooses 5 triples via random walk [36] starting from topic entities; 4) **5 random triples** selects 5 triples from the n-hop subgraph; and 5) **LLM relevance pruning** utilizes a standard LLM to filter triples strictly based on semantic relevance without preference alignment. Overall, these alternatives lead to lower performance than KD-GAG. Heuristic methods such as BM25, random walk, and random sampling perform poorly on most datasets, suggesting that simple relevance or structure-based selection may fail to preserve key reasoning evidence. Although BM25 and random walk outperform the full n-hop subgraph on MedHop, they still lag behind KD-GAG, indicating that concise evidence selection alone is insufficient. In addition, LLM relevance pruning performs better than most heuristic methods but remains inferior to the optimized retriever, with 5.4 percentage points lower EM on NQ and 1.9 percentage points lower accuracy on MedHop. This suggests that the gain of KD-GAG is not merely due to removing irrelevant triples, but also comes from aligning the selected evidence with the downstream generator’s preferences.

Moreover, we conducted joint ablation experiments on both modules by using the Student LLM for knowledge graph construction and employing an unoptimized LLM as the retriever to filter the n-hop subgraph. Compared with the setup where the Student LLM constructs the knowledge graph with the optimized retriever, this configuration further degrades performance on all four datasets. For example, EM decreases by 4.4 and 3.6 percentage points

Table 2: Ablation results of KD-GAG on HotpotQA, 2WikiMQA, NQ, and MedHop datasets.

Method	Details	HotpotQA		2WikiMQA		NQ		MedHop
		EM	F1	EM	F1	EM	F1	Acc
w/o distillation	KGC by Student LLM	44.6	57.8	44.2	46.9	33.6	37.9	17.1
w/o optimized retriever	N-hop subgraph	53.0	58.7	41.8	44.0	46.4	50.5	14.3
	Top-5 triples by BM25	36.6	41.3	27.0	29.2	37.6	41.7	17.0
	5 triples by random walk	27.4	31.4	26.8	29.3	31.9	35.9	15.8
	5 random triples	28.2	32.4	27.4	30.1	27.1	31.7	8.2
	LLM relevance pruning	47.6	53.5	43.2	45.6	45.4	49.7	17.7
w/o distillation and optimized retriever	KGC by Student LLM + LLM relevance pruning	42.6	52.2	39.8	44.7	30.0	34.8	16.5
Ours		59.8	67.0	56.6	58.6	50.8	54.9	19.6

on 2WikiMQA and NQ, respectively, while F1 decreases by 5.6 percentage points on HotpotQA. MedHop accuracy also drops from 17.1 to 16.5. Since both settings use the same Student-constructed knowledge graphs, the additional performance gap can be mainly attributed to the retrieval module. These results show that an unoptimized retriever may introduce noisy triples or overlook critical evidence, whereas our preference-based optimization helps select knowledge more suitable for the generator’s reasoning process.

5.4. Efficiency and Quality Analysis of Knowledge Distillation (RQ3)

To address RQ3, which investigates whether the distilled Student LLM can significantly enhance construction efficiency while maintaining high-quality knowledge extraction, we designed a comprehensive comparative experiment. Given the lack of standard triple extraction benchmarks specifically tailored for GraphRAG scenarios, we constructed a high-quality, human-verified test set based on HotpotQA. Specifically, we randomly sampled 500 context chunks from the HotpotQA dataset and employed GPT-4o to perform initial triple extraction on these fragments. Subsequently, we introduced a rigorous human verification process, where professional annotators corrected the initial results by rectifying erroneous entities, completing missing relations, and removing semantic redundancies. The final verified triples served as the ground truth for this evaluation. On this basis, we comprehensively evaluated both the multi-stage Teacher LLM and the end-to-end Student LLM across two dimensions: extraction quality (measured by Precision, Recall, and F1 Score) and computational efficiency (measured by Inference Latency and Number of API Calls). Notably, we adopt a semantic-based evaluation protocol where an extracted triple is considered a true positive if its elements

Table 3: Comparison of extraction quality and efficiency on 500 randomly sampled HotpotQA contexts. Latency is measured in seconds per text chunk.

Model	Efficiency		Extraction Quality (%)		
	Latency (s) ↓	Calls ↓	Precision	Recall	F1
Teacher LLM (Multi-Stage)	85.62	6	97.24	90.97	94.00
Student LLM (End-to-End)	3.42	1	99.86	86.40	92.65

(*head, relation, tail*) are semantically equivalent to the ground truth. This approach effectively avoids the underestimation typically caused by strict string matching.

The empirical results in Table 3 reveal that KD-GAG achieves a highly favorable trade-off between computational efficiency and extraction fidelity, validating the efficacy of our end-to-end distillation strategy. The most substantial gain lies in computational efficiency. The Teacher LLM, constrained by its multi-stage pipeline, suffers from a heavy inference bottleneck, requiring 6 API calls per text chunk and averaging 85.62 seconds. In contrast, the Student LLM internalizes this complex reasoning process into a single-pass generation. This architectural simplification reduces the API calls to 1 and reduces the inference latency to 3.42 seconds, representing an approximate $25\times$ speedup. This confirms that KD-GAG effectively shifts the computational burden from the inference phase to the training phase, making large-scale graph construction computationally feasible.

Crucially, this efficiency gain does not come at the cost of significant performance degradation. Despite the reduction in model size, the Student LLM maintains a competitive F1 score of 92.65%, closely trailing the Teacher model (94.00%). A notable and counter-intuitive finding is that the Student model actually outperforms the Teacher in terms of Precision (99.86% vs. 97.24%). We attribute this to the denoising effect inherent in supervised fine-tuning: trained on high-quality, verified data, the Student LLM tends to be more conservative and rigorous, thereby mitigating the generation of hallucinated entities. While this conservative behavior results in a minor drop in Recall ($\sim 4.6\%$), in the context of GraphRAG, high-precision triples are often prioritized to minimize noise propagation during the retrieval phase.

5.5. Effectiveness of retrieved triples (RQ_4)

To evaluate retrieval quality in the absence of triple-level annotations, we constructed a pseudo-labeled evaluation set by prompting GPT-5.5 with

the query, gold answer, and our constructed knowledge graph to extract the minimal essential triples. Considering the high API costs associated with extensive LLM invocations, we randomly sampled 1,000 queries from HotpotQA and 1,000 queries from NQ for this specific evaluation. Using these extracted triples as proxy ground truth, we assessed retrieval performance via Recall and Answer Hit Rate on HotpotQA and NQ. Recall measures the proportion of pseudo-labeled essential triples that are retrieved, and Answer Hit Rate measures whether the retrieved triples contain the gold answer or its normalized alias. We compared our preference-optimized retriever against two baselines: BM25 (selecting top-5 triples) and a standard LLM-based relevance pruner, which represents semantic denoising without explicit preference alignment.

As shown in Table 4, our preference-optimized retriever consistently outperforms the baselines across both datasets. Notably, comparing our method to the LLM relevance pruning baseline helps disentangle the impact of preference alignment from simple noise reduction. While the standard LLM baseline filters general noise based on semantic similarity, it struggles to capture the specific reasoning requirements of the downstream generator. In contrast, our approach achieves a 7.0% and 5.3% improvement in Recall on HotpotQA and NQ, respectively, alongside notable gains in Answer Hit Rate. This demonstrates that DPO-based alignment encourages the retriever to retain triples that are not just broadly relevant, but specifically preferred by the generator for accurate inference. The significant performance gap provides empirical evidence that the gain cannot be fully explained by generic denoising alone, and that preference-aware retriever-generator alignment contributes additional benefits.

Table 4: Evaluation of retrieval quality on HotpotQA and NQ datasets.

Method	HotpotQA		NQ	
	Recall	Answer Hit Rate	Recall	Answer Hit Rate
Top-5 triples by BM25	32.3	40.0	23.1	45.0
LLM relevance pruning	48.8	52.6	39.7	60.0
Ours	55.8	56.7	45.0	61.3

5.6. Student LLM Selection in Knowledge Distillation (RQ5)

We investigate the impact of Student LLM scale on KD-GAG’s performance on HotpotQA and 2WikiMQA, and the experimental results are pre-

sented in Fig 3. As expected, a larger LLM (Qwen2.5-7B-Instruct) yields higher-quality knowledge graphs and consequently better QA performance on both datasets. Notably, even the smallest LLM (Qwen2.5-0.5B-Instruct) achieves competitive results, achieving F1 scores of 66.3 on HotpotQA and 55.0 on 2WikiMQA, demonstrating that our distillation approach effectively enhances small LLMs’ performance. Thus, KD-GAG enables effective knowledge extraction even with limited computational resources, positioning it as a practical solution for resource-limited deployments.



Figure 3: Performance comparison of student LLMs (Qwen2.5-{7, 3, 1.5, 0.5}B-Instruct).

5.7. Cross-Generator Generalization Analysis (RQ6)

To analyze how strongly the learned pruner depends on the fixed generator used in reward computation, we conducted a cross-generator generalization experiment on the HotpotQA dataset. We kept the retriever fixed after optimizing it with answer-level feedback from the default generator, Qwen2.5-7B-Instruct, and evaluated the end-to-end QA performance when this fixed retriever was paired with unseen generators during inference.

As shown in Table 5, the learned retriever is not narrowly overfitted to the exact generator used for reward computation. When paired with Qwen3-8B and Qwen2.5-3B-Instruct, the system still achieves competitive F1 scores of 64.6 and 60.9, respectively, compared with 67.0 using the default Qwen2.5-7B-Instruct generator. This indicates that the pruner learns transferable relevance and denoising patterns that generalize across Qwen-family generators with different model scales and versions.

However, when the generator is replaced with Llama-3-8B-Instruct, the performance drops to an F1 score of 49.9. This suggests that cross-family

transfer may be more challenging. We note that this drop should not be attributed solely to retriever-generator mismatch, because the final QA performance is also affected by the intrinsic reasoning and instruction-following capability of each generator. Therefore, this experiment should be interpreted as an end-to-end cross-generator robustness analysis rather than a fully isolated measurement of retriever transferability. Overall, the results show that our pruner is not narrowly overfitted to the exact reward generator, while lightweight re-alignment may still be beneficial when transferring to substantially different model families.

Table 5: Cross-generator generalization performance on HotpotQA using the retriever optimized by Qwen2.5-7B-Instruct.

Generator	HotpotQA	
	EM	F1
Qwen2.5-7B-Instruct (ours)	59.8	67.0
Qwen3-8B	58.6	64.6
Qwen2.5-3B-Instruct	55.6	60.9
Llama-3-8B-Instruct	47.4	49.9

6. Limitations

Preference-aware alignment introduces a degree of generator dependence into the pruning retriever. Our cross-generator results suggest that the optimized retriever can transfer to closely related generators, whereas substantially different generator families may reduce this transferability. Importantly, replacing the generator does not require reconstructing the knowledge graph or retraining the Student LLM. The existing retriever can first be reused and validated with the new generator. When substantial degradation is observed, lightweight re-alignment can be limited to the pruning retriever using answer-level preference feedback from the new generator, rather than retraining the entire KD-GAG pipeline. This represents an important deployment tradeoff of the proposed preference-aware alignment strategy.

7. Conclusion

In this paper, we proposed a novel framework, KD-GAG, to address the limitations of existing GraphRAG methods. By leveraging knowledge distil-

lation, KD-GAG efficiently constructs high-quality knowledge graphs while reducing computational costs. Additionally, a reinforcement learning-based approach optimizes subgraph pruning, aligning retriever selections with generator preferences and minimizing irrelevant information. Extensive experimental results across diverse benchmarks, including HotpotQA, 2WikiMQA, NQ, and MedHop, demonstrate the state-of-the-art performance and robust generalizability of our method across different domains and reasoning complexities. Our future work will focus on exploring multi-modal extensions and more complex reasoning tasks. Through expanding the capabilities of KD-GAG to handle diverse data types and intricate reasoning requirements, we aim to further enhance its applicability and practical value in real-world scenarios.

8. Acknowledgement

This work is Funded by Anhui Postdoctoral Scientific Research Program Foundation(No.2024A768).

Appendix A. Prompt Templates

In this section, we provide the detailed prompt templates used in KD-GAG. The variables enclosed in curly braces (e.g., {text}) are placeholders replaced by actual data during inference.

Appendix A.1. Knowledge Graph Construction

The Knowledge Graph Construction process differs between the Teacher and Student models.

- **Teacher LLM (Multi-Stage):** To ensure high-quality training data, the Teacher LLM follows a rigorous pipeline consisting of six stages (Stages 1–6 below), including entity extraction, merging, relation definition, and optimization.
- **Student LLM (End-to-End):** The Student LLM is distilled to perform knowledge extraction in a single pass. Therefore, during inference, the Student LLM **only utilizes the prompt from Stage 1 (Initial Triple Extraction)**, effectively bypassing the subsequent intermediate stages (Stages 2–6) while maintaining high extraction quality.

Stage 1: Initial Triple Extraction

Your task is to transform the given text into a semantic graph in the form of a list of triples. The triples must be in the form of [Entity1, Relationship, Entity2]. In your answer, please strictly only include the triples and do not include any explanation or apologies.

Here are some examples: {examples}

Now please extract triplets from the following text.

Topic: {topic}

Text: {text}

Triples:

Stage 2: Entity Extraction

Your task is to extract entities from the given text. In your answer, please strictly only include the entities and do not include any explanation or apologies.

Here are some examples: {examples}

Now please extract entities from the following text.

Topic: {topic}

Text: {text}

Entities:

Stage 3: Entity Merging

Given a piece of text and two lists of entities extracted from it, merge them into one single list, make sure there is no redundant or duplicate entities in your answer.

Text: {text}

Entity list 1: {entity_list_1}

Entity list 2: {entity_list_2}

Answers:

Stage 4: Relation Definition

You will be given a piece of text and a list of relational triples in the format of [Subject, Relation, Object] extracted from the text. For each relation present in the triples, your task is to write a description to express the meaning of the relation.

Here are some examples: {examples}

Now please extract relation descriptions given the following text and triples. Note that the description needs to be general and can be used to describe relations between other entities as well. Pay attention to the order of subject and object entities.

Text: {text}

Triples: {extracted_triples}

Relations: {extracted_relations}

Answer:

Stage 5: Relation Optimization

Given the following text and a relational triplet extracted from it:

Text: {text}

Triplet: {query_triplet}

The relation '{query_relation}' in the triplet is defined as '{query_relation_definition}' In this context, is there any relation appropriate to replace it? Please answer by providing only the letter of your choice!

Choices: {choices}

answer:

Stage 6: Final Extraction

Your task is to transform the given text into a semantic graph in the form of a list of triples. The triples must be in the form of [Entity1, Relationship, Entity2]. In your answer, please strictly only include the triples and do not include any explanation or apologies.

Here are some examples: {examples}

Now please extract triplets from the following text. Here are some potential relations and their descriptions you may look out for during extraction:

{relations}

Note that this list may not be exhaustive, you may use other relations and not necessarily all relations in this list are present in the text.

Text: {text}

Candidate entities: {entities}

Appendix A.2. Retriever Optimization (Subgraph Pruning)

The following prompt is used for the pruning process, where the retriever selects relevant triples to form the optimized subgraph.

Subgraph Pruning

You are provided with a list of triples in the format [subject, relation, object]. Your task is to extract and return only the triples that are relevant to answering the following question. The extracted triples must come exactly from the provided list without any modification—do not change, add, or remove any words or punctuation. Only include triples that contribute direct or indirect support for answering the question. Output nothing but the relevant triple(s) in their original format and do not include any explanation or apologies.

Question: {question}

Triples: {subgraph}

Relevant triples:

Appendix A.3. Graph-Augmented Generation

Finally, the generator uses the pruned subgraph to answer the user query.

Final Question Answering

Please answer the question based on the knowledge from the following triplets, provide only the answer, and do not include any explanation or apologies. If you cannot infer the answer to the question from the provided triples, output 'unknown':

Question: {question}

Triplets: {pruned_subgraph}

Answer:

References

- [1] S. Es, J. James, L. E. Anke, S. Schockaert, Ragas: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 2024, pp. 150–158.
- [2] A. Salemi, H. Zamani, Evaluating retrieval quality in retrieval-augmented generation, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2024, pp. 2395–2400.
- [3] R. Hong, H. Zhang, H. Zhao, D. Yu, C. Zhang, Faithful question answering with monte-carlo planning, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 3944–3965.
- [4] X. Ma, Y. Gong, P. He, H. Zhao, N. Duan, Query rewriting in retrieval-augmented large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 5303–5315.
- [5] Z. Xu, Z. Liu, Y. Liu, C. Xiong, Y. Yan, S. Wang, S. Yu, Z. Liu, G. Yu, Activerag: Revealing the treasures of knowledge via active learning, arXiv e-prints (2024) arXiv–2402.

- [6] H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal, Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 10014–10037.
- [7] C. Huang, F. Yu, Z. Wan, F. Li, H. Ji, Y. Li, Knowledge graph confidence-aware embedding for recommendation, *Neural Networks* 180 (2024) 106601.
- [8] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, *arXiv preprint arXiv:2310.01061* (2023).
- [9] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph, *arXiv preprint arXiv:2307.07697* (2023).
- [10] Z. Guo, L. Xia, Y. Yu, T. Ao, C. Huang, Lightrag: Simple and fast retrieval-augmented generation, *arXiv e-prints* (2024) arXiv:2410.11429.
- [11] T. Fan, J. Wang, X. Ren, C. Huang, Minirag: Towards extremely simple retrieval-augmented generation, *arXiv preprint arXiv:2501.06713* (2025).
- [12] Y. Wang, N. Lipka, R. A. Rossi, A. Siu, R. Zhang, T. Derr, Knowledge graph prompting for multi-document question answering, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 19206–19214.
- [13] Z. Liu, Q. Cao, N. Du, H. Shu, E. Zhong, N. Jiang, Q. Chen, Y. Shen, K. Chen, Fit-graph: A multi-grained evolutionary graph based framework for disease diagnosis, *Artificial Intelligence in Medicine* 147 (2024) 102735.
- [14] C. Wang, W. Deng, W. Guan, Q. Lu, N. Jiang, Cross-granularity hypergraph retrieval-augmented generation for multi-hop question answering, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 33368–33376.

- [15] L. Zhuang, S. Chen, Y. Xiao, H. Zhou, Y. Zhang, H. Chen, Q. Zhang, X. Huang, Linearrag: Linear graph retrieval augmented generation on large-scale corpora, arXiv preprint arXiv:2510.10114 (2025).
- [16] J. Guo, J. Liu, D. Xu, Jointpruning: Pruning networks along multiple dimensions for efficient point cloud processing, *IEEE Transactions on Circuits and Systems for Video Technology* 32 (6) (2021) 3659–3672.
- [17] J. Guo, J. Liu, D. Xu, 3d-pruning: A model compression framework for efficient 3d action recognition, *IEEE Transactions on Circuits and Systems for Video Technology* 32 (12) (2022) 8717–8729.
- [18] J. Gou, B. Yu, S. J. Maybank, D. Tao, Knowledge distillation: A survey, *International Journal of Computer Vision* 129 (6) (2021) 1789–1819.
- [19] J. Chung, E. Kamar, S. Amershi, Increasing diversity while maintaining accuracy: Text data generation with large language models and human interventions, in: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 575–593.
- [20] X. Li, P. Yu, C. Zhou, T. Schick, O. Levy, L. Zettlemoyer, J. Weston, M. Lewis, Self-alignment with instruction backtranslation, arXiv preprint arXiv:2308.06259 (2023).
- [21] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, *Advances in neural information processing systems* 35 (2022) 27730–27744.
- [22] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, D. Amodei, Deep reinforcement learning from human preferences, *Advances in neural information processing systems* 30 (2017).
- [23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017).
- [24] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, C. Finn, Direct preference optimization: Your language model is secretly a reward model, *Advances in Neural Information Processing Systems* 36 (2023) 53728–53741.

- [25] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al., Training a helpful and harmless assistant with reinforcement learning from human feedback, arXiv preprint arXiv:2204.05862 (2022).
- [26] F. Liu, et al., Learning to summarize from human feedback, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 583–592.
- [27] X. V. Lin, X. Chen, M. Chen, W. Shi, M. Lomeli, R. James, P. Rodriguez, J. Kahn, G. Szilvassy, M. Lewis, et al., Ra-dit: Retrieval-augmented dual instruction tuning, in: The Twelfth International Conference on Learning Representations, 2023.
- [28] B. Zhang, H. Soh, Extract, define, canonicalize: An llm-based framework for knowledge graph construction, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 9820–9836.
- [29] A. Bundy, L. Wallen, Breadth-first search, Catalogue of artificial intelligence tools (1984) 13–13.
- [30] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, C. D. Manning, Hotpotqa: A dataset for diverse, explainable multi-hop question answering, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 2369–2380.
- [31] X. Ho, A.-K. D. Nguyen, S. Sugawara, A. Aizawa, Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps, in: Proceedings of the 28th International Conference on Computational Linguistics, 2020, pp. 6609–6625.
- [32] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, et al., Natural questions: a benchmark for question answering research, Transactions of the Association for Computational Linguistics 7 (2019) 453–466.
- [33] J. Welbl, P. Stenetorp, S. Riedel, Constructing datasets for multi-hop reading comprehension across documents, Transactions of the Association for Computational Linguistics 6 (2018) 287–302.

- [34] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al., Lora: Low-rank adaptation of large language models., ICLR 1 (2) (2022) 3.
- [35] S. Robertson, H. Zaragoza, et al., The probabilistic relevance framework: Bm25 and beyond, Foundations and Trends® in Information Retrieval 3 (4) (2009) 333–389.
- [36] F. Xia, J. Liu, H. Nie, Y. Fu, L. Wan, X. Kong, Random walks: A review of algorithms and applications, IEEE Transactions on Emerging Topics in Computational Intelligence 4 (2) (2019) 95–107.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.